

Funções hash: origem, motivação e evolução histórica

Hash functions: origin, motivation, and historical evolution

Prof. Ms. José Augusto Navarro Garcia Manzano - ORCID: 0000-0001-9248-7765

augusto.garcia@ifsp.edu.br

Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP)

<https://zenodo.org/records/20395094> - DOI: 10.5281/zenodo.20395094

Resumo

As funções hash são mecanismos fundamentais da computação contemporânea, amplamente empregados tanto em estruturas de dados quanto em sistemas criptográficos. Seu uso abrange indexação eficiente, recuperação rápida de informações, verificação de integridade, autenticação e assinaturas digitais. Historicamente, o conceito surgiu da necessidade de reduzir o custo computacional associado à busca e ao armazenamento de dados em grandes conjuntos de informação. As primeiras ideias relacionadas ao hashing são frequentemente associadas aos trabalhos de Hans Peter Luhn, na década de 1950, especialmente no contexto da recuperação automática de informação. Posteriormente, o conceito foi aprofundado e ganhou importância criptográfica com os trabalhos de Ralph Merkle, no final da década de 1970, especialmente no desenvolvimento de funções hash unidirecionais. Este artigo apresenta a origem histórica das funções hash, suas motivações iniciais, sua evolução conceitual e exemplos práticos de implementação em C, C++ e Python.

Palavras-chave: funções hash, hashing, estruturas de dados, criptografia, evolução histórica.

Abstract

Hash functions are fundamental mechanisms in contemporary computing, widely employed in both data structures and cryptographic systems. Their use encompasses efficient indexing, fast information retrieval, integrity verification, authentication, and digital signatures. Historically, the concept emerged from the need to reduce the computational cost associated with searching and storing data in large information sets. The earliest ideas related to hashing are often associated with the work of Hans Peter Luhn in the 1950s, particularly in the context of automatic information retrieval. Later, the concept was further developed and gained cryptographic significance through the work of Ralph Merkle in the late 1970s, especially in the development of one-way hash functions. This article presents the historical origin of hash functions, their initial motivations, their conceptual evolution, and practical implementation examples in C, C++, and Python.

Keywords: hash functions, hashing, data structures, cryptography, historical evolution.

1. Introdução

Funções hash são algoritmos destinados a transformar uma entrada de tamanho arbitrário em uma saída de tamanho fixo. Em ciência da computação, essas funções desempenham papel central em estruturas de dados, sistemas de armazenamento, bancos de dados, compiladores e aplicações criptográficas. Knuth (1998) discute o endereçamento por dispersão como uma técnica essencial para a organização eficiente de dados, enquanto Cormen et al. (2022) destacam a relevância das funções hash em algoritmos e estruturas de dados modernas.

No contexto de estruturas de dados, funções hash permitem mapear chaves diretamente para posições de armazenamento, reduzindo significativamente o custo de busca. Em aplicações criptográficas, são utilizadas para verificação de integridade, autenticação de mensagens, armazenamento seguro de senhas e implementação de assinaturas digitais.

A principal característica dessas funções é a capacidade de representar grandes volumes de informação por meio de resumos compactos, preservando propriedades relevantes para identificação, recuperação ou validação de dados.

Historicamente, o conceito de hashing surgiu associado à necessidade de acelerar mecanismos de busca em sistemas computacionais. Trabalhos pioneiros desenvolvidos por Hans Peter Luhn (1953) propuseram métodos de organização e indexação de informações que anteciparam princípios fundamentais das tabelas hash contemporâneas.

Posteriormente, a evolução das técnicas criptográficas ampliou significativamente a importância dessas funções. Em 1979, Ralph Merkle consolidou o estudo de funções hash unidirecionais em sua tese *Secrecy, Authentication, and Public Key Systems*, estabelecendo fundamentos importantes para a criptografia moderna.

2. Motivação histórica

A motivação original das funções hash não estava relacionada à criptografia, mas à eficiência computacional.

Durante as décadas de 1950 e 1960, os sistemas computacionais operavam predominantemente com armazenamento sequencial em cartões perfurados, fitas magnéticas e memórias de acesso limitado. A recuperação de informações dependia, em muitos casos, de buscas lineares, cuja complexidade temporal crescia proporcionalmente ao número de elementos armazenados.

À medida que o volume de dados aumentava, esse modelo tornou-se inadequado para aplicações científicas, administrativas e comerciais. Nesse contexto, surgiu a necessidade de desenvolver mecanismos capazes de localizar informações de forma mais eficiente.

A ideia fundamental do *hashing* consistiu em aplicar uma transformação matemática diretamente sobre a chave de busca, produzindo um índice capaz de indicar a posição provável de armazenamento do dado. Dessa forma, evitava-se a necessidade de percorrer integralmente o conjunto de registros.

Formalmente, uma função hash pode ser representada como:

$$h(k) = k \bmod m$$

em que:

- k representa a chave;
- m representa o tamanho da tabela de armazenamento.

Essa abordagem permitiu reduzir o custo médio de busca para tempo aproximadamente constante.

No contexto criptográfico, entretanto, a motivação passou a ser distinta. O objetivo deixou de ser apenas velocidade de acesso e passou a incluir propriedades de segurança, como:

- resistência à colisão;
- resistência à pré-imagem;
- autenticação de mensagens;
- verificação de integridade.

Os trabalhos de Ralph Merkle foram decisivos nesse processo de transição conceitual.

3. Surgimento e evolução das funções hash

Embora não exista consenso absoluto sobre um único inventor das funções hash, a literatura especializada reconhece Hans Peter Luhn como um dos primeiros pesquisadores a empregar conceitos equivalentes ao *hashing* moderno em sistemas de recuperação de informação.

Em memorandos internos da IBM datados de 1953, Luhn (1953) descreveu métodos de organização de registros baseados em agrupamentos, antecipando princípios importantes das tabelas hash contemporâneas.

Poucos anos depois, em 1956, Arnold Dumey publicou um dos primeiros trabalhos formais descrevendo técnicas de espalhamento utilizando o operador módulo, método ainda amplamente empregado em implementações modernas.

Na década de 1960, o hashing tornou-se amplamente difundido em compiladores, sistemas operacionais, bancos de dados e recuperação de informação. Nesse contexto, Morris (1968) descreveu técnicas de *scatter storage* para entrada e recuperação de informações em tabelas hash, contribuindo para o desenvolvimento de métodos mais eficientes de endereçamento..

A partir do final da década de 1970, iniciou-se a consolidação das funções hash criptográficas. Nesse período, Ralph Merkle formalizou o conceito de função hash unidirecional, estabelecendo bases para sistemas de autenticação e assinaturas digitais.

Essa evolução permitiu o desenvolvimento posterior de algoritmos criptográficos amplamente conhecidos, como:

- MD5;
- SHA-1;
- SHA-256.

4. Importância criptográfica

As funções hash criptográficas desempenham papel essencial na segurança da informação moderna. Sua relevância decorre principalmente de três propriedades fundamentais:

- resistência à pré-imagem;
- resistência à segunda pré-imagem;
- resistência a colisões.

Essas propriedades garantem que pequenas alterações na mensagem original produzam resumos significativamente diferentes, dificultando a falsificação ou reconstrução da informação original.

Estudos consolidados na área de segurança, como os de Stinson e Paterson (2018), discutem as funções hash como mecanismos de resumo e dispersão aplicados à segurança da informação, destacando seu uso em integridade, autenticidade e validação de dados.

Em termos práticos, funções hash criptográficas são utilizadas em:

- assinaturas digitais;
- certificados digitais;
- protocolos de autenticação;
- armazenamento seguro de senhas;
- blockchain;
- mecanismos HMAC.

O princípio geral consiste em transformar mensagens extensas em resumos compactos, frequentemente denominados "impressões digitais" digitais (*digital fingerprints*).

5. Exemplos práticos

Os exemplos a seguir utilizam uma adaptação didática do algoritmo DJB2, desenvolvida por Daniel J. Bernstein. O objetivo é apenas ilustrar o funcionamento básico de uma função hash aplicada a cadeias de caracteres.

O algoritmo utiliza deslocamentos binários à esquerda:

$(h \ll 5)$

equivale a multiplicar o valor de h por 2^5 , isto é, 32. Assim, a expressão:

$(h \ll 5) + h$

equivale matematicamente a:

$32 \cdot h + h = 33 \cdot h$

produzindo o fator multiplicativo 33, característico do algoritmo DJB2.

5.1 Implementação em C

Em C, a implementação opera diretamente sobre cadeias de caracteres representadas por ponteiros.

```
#include <stdio.h>
#include <stdint.h>

uint32_t hash_string(const char *s) {
    uint32_t h = 5381;

    while (*s) {
```

```

        h = ((h << 5) + h) + (unsigned char)(*s);
        s++;
    }

    return h;
}

int main(void) {
    const char *texto = "funcao hash";

    printf("%u\n", hash_string(texto));

    return 0;
}

```

5.2 Implementação em C++

Em C++, a mesma lógica pode ser aplicada utilizando objetos do tipo **std::string**.

```

#include <iostream>
#include <string>
#include <cstdint>

uint32_t hash_string(const std::string& s) {
    uint32_t h = 5381;

    for (unsigned char c : s) {
        h = ((h << 5) + h) + c;
    }

    return h;
}

int main(void) {
    std::string texto = "funcao hash";

    std::cout << hash_string(texto) << std::endl;

    return 0;
}

```

A linguagem também fornece o objeto funcional **std::hash**, definido na biblioteca **<functional>** e utilizado em estruturas associativas como **std::unordered_map** e **std::unordered_set**.

```

#include <iostream>
#include <string>
#include <functional>

int main()
{
    std::string texto = "funcao hash";

    std::hash<std::string> hash_string;

    size_t resultado = hash_string(texto);

    std::cout << "Texto: " << texto << std::endl;
    std::cout << "Hash: " << resultado << std::endl;

    return 0;
}

```

5.3 Implementação em Python

Em Python, o algoritmo pode ser reproduzido manualmente.

```
def hash_string(s: str) -> int:
    h = 5381

    for c in s:
        h = ((h << 5) + h) + ord(c)

    return h

print(hash_string("funcao hash"))
```

A linguagem Python disponibiliza a função nativa **hash()**, utilizada em estruturas como **dict** e **set**.

```
def hash_string(s: str) -> int:
    return hash(s)

texto = "funcao hash"

print(hash_string(texto))
```

Em versões modernas da linguagem, os valores produzidos por **hash()** podem variar entre execuções devido ao mecanismo interno de randomização conhecido como *hash salting*.

6. Conclusão

As funções hash surgiram inicialmente como soluções para problemas de indexação e recuperação eficiente de informações em grandes conjuntos de dados. Sua evolução histórica demonstra uma transição significativa: de técnicas voltadas à otimização de estruturas de armazenamento para mecanismos essenciais da criptografia contemporânea.

Os trabalhos pioneiros de Hans Peter Luhn (1953) contribuíram para os fundamentos do *hashing* aplicado à recuperação de informação, enquanto as contribuições de Ralph Merkle (1979) consolidaram o uso criptográfico das funções hash modernas.

Atualmente, essas funções desempenham papel central em praticamente todas as áreas da computação, incluindo:

- estruturas de dados;
- sistemas distribuídos;
- segurança da informação;
- blockchain;
- autenticação digital;
- bancos de dados;
- compiladores.

Sua importância decorre da combinação entre eficiência computacional, capacidade de compactação e propriedades matemáticas adequadas para aplicações de desempenho e segurança.

Referências

CORMEN, Thomas H. et al. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.

DUMEY, A. I. Indexing for rapid random access memory systems. Computers and Automation, v. 5, n. 12, p. 6-9, dec. 1956.

KNUTH, Donald E. The Art of Computer Programming: Sorting and Searching. 2. ed. Reading: Addison-Wesley, 1998.

LUHN, H. P. A new method of recording and searching information. American Documentation, v. 4, n. 1, p. 14-16, jan. 1953. DOI: <https://doi.org/10.1002/asi.5090040104>. Disponível em: <https://onlinelibrary.wiley.com/doi/epdf/10.1002/asi.5090040104>. Acesso em: 25 maio 2026.

MERKLE, Ralph Charles. Secrecy, authentication, and public key systems. 1979. 194 p. Thesis (Ph.D. in Electrical Engineering) – Department of Electrical Engineering, Stanford University, Stanford, CA, 1979. Technical Report No. 1979-1. Disponível em: <https://www.ralphmerkle.com/papers/Thesis1979.pdf>. Acesso em: 25 maio 2026.

MORRIS, R. Scatter storage techniques. Communications of the ACM, v. 11, n. 1, p. 38-44, jan. 1968. DOI: [doi.org](https://doi.org/10.1145/362851.362882). Disponível em: <https://dl.acm.org/doi/epdf/10.1145/362851.362882>. Acesso em: 25 maio 2026.

STINSON, D. R.; PATERSON, M. B. Cryptography: Theory and Practice. 4. ed. Boca Raton: Chapman and Hall/CRC, 2018.